# CSE 332 Spring 2026
# Lecture 8: AVL Trees

Nathan Brunelle

http://www.cs.uw.edu/332

# Dictionary (Map) ADT

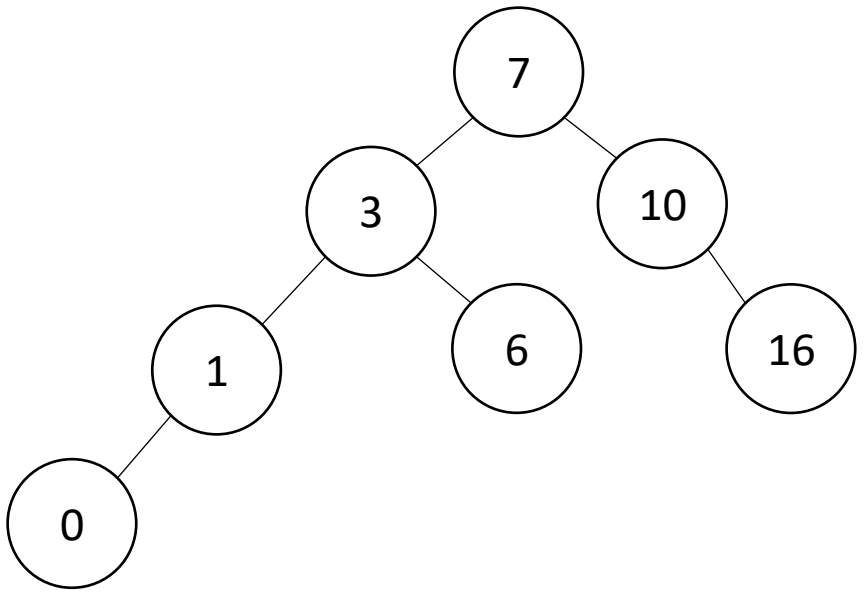
- Contents:
  - Sets of key+value pairs
  - Keys must be comparable
- Operations:
  - insert(key, value)
    - Adds the (key,value) pair into the dictionary
    - If the key already has a value, overwrite the old value
      - Consequence: Keys cannot be repeated
  - find(key)
    - Returns the value associated with the given key
  - delete(key)
    - Remove the key (and its associated value)

# Naïve attempts

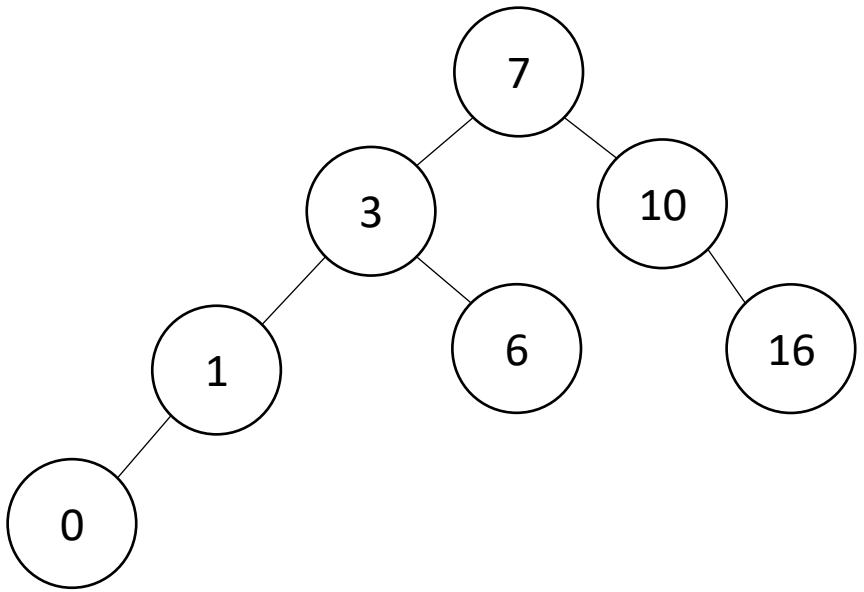
| Data Structure | Time to insert | Time to find | Time to delete |
|---|---|---|---|
| Unsorted Array | $\Theta(n)$ | $\Theta(n)$ | $\Theta(n)$ |
| Unsorted Linked List | $\Theta(n)$ | $\Theta(n)$ | $\Theta(n)$ |
| Sorted Array | $\Theta(n)$ | $\Theta(\log n)$ | $\Theta(n)$ |
| Sorted Linked List | $\Theta(n)$ | $\Theta(n)$ | $\Theta(n)$ |
| Heap | $\Theta(n)$ | $\Theta(n)$ | $\Theta(n)$ |
| Binary Search Tree | $\Theta(n)$ | $\Theta(n)$ | $\Theta(n)$ |
| AVL Tree | $\Theta(\log n)$ | $\Theta(\log n)$ | $\Theta(\log n)$ |

# Find Operation (recursive)

```
find(key, root){
        if (root == Null){
                return Null;
        {
        if (key == root.key){
                return root.value;
        }
        if (key < root.key){
                return find(key, root.left);
        }
        if (key > root.key){
                return find(key, root.right);
        }
        return Null;
}
```



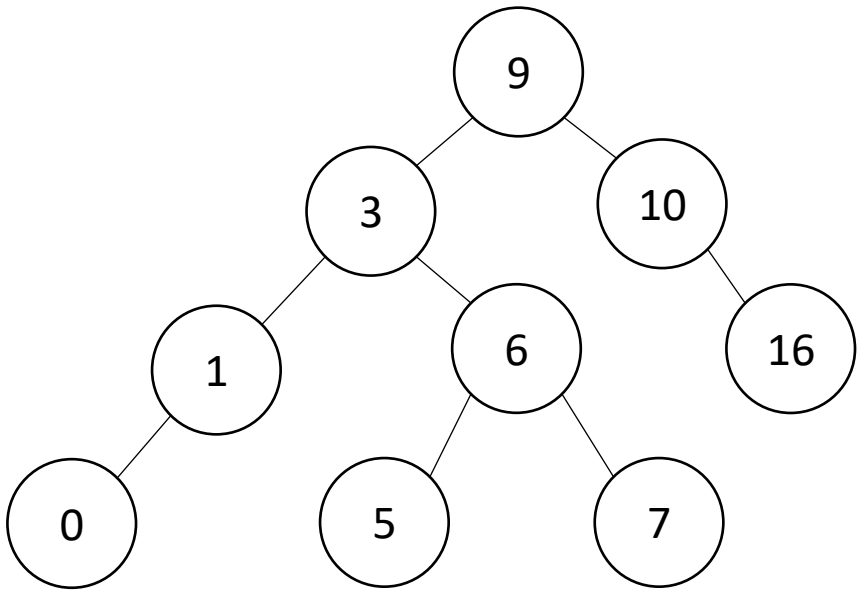
# Insert Operation (recursive)




```
insert(key, value, root){
        root = insertHelper(key, value, root);
}
insertHelper(key, value, root){
        if(root == null)
                return new Node(key, value);
        if (root.key < key)
                root.right = insertHelper(key, value, root.right);
        else
                root.left = insertHelper(key, value, root.left);
        return root;
}
```

Note: Insert happens only at the leaves!

# Delete Operation (recursive)




```
delete(key, root){
    if (root == Null){ return; } // key not present
    if (root.key == key){
        if (root has no children) { return Null; }
        if (root has one child) { return that child; }
        if (root has two children) {return removeMax(root.left);}
    }
    if (root.key < key) { root.right = delete(key, root.right); }
    else { root.left = delete(key, root.left); }
}
```

# Worst Case Analysis

- For each of Find, insert, Delete:
  - Worst case running time matches height of the tree
- What is the maximum height of a BST with $n$ nodes?
  - $\Theta(n)$

# Improving the worst case

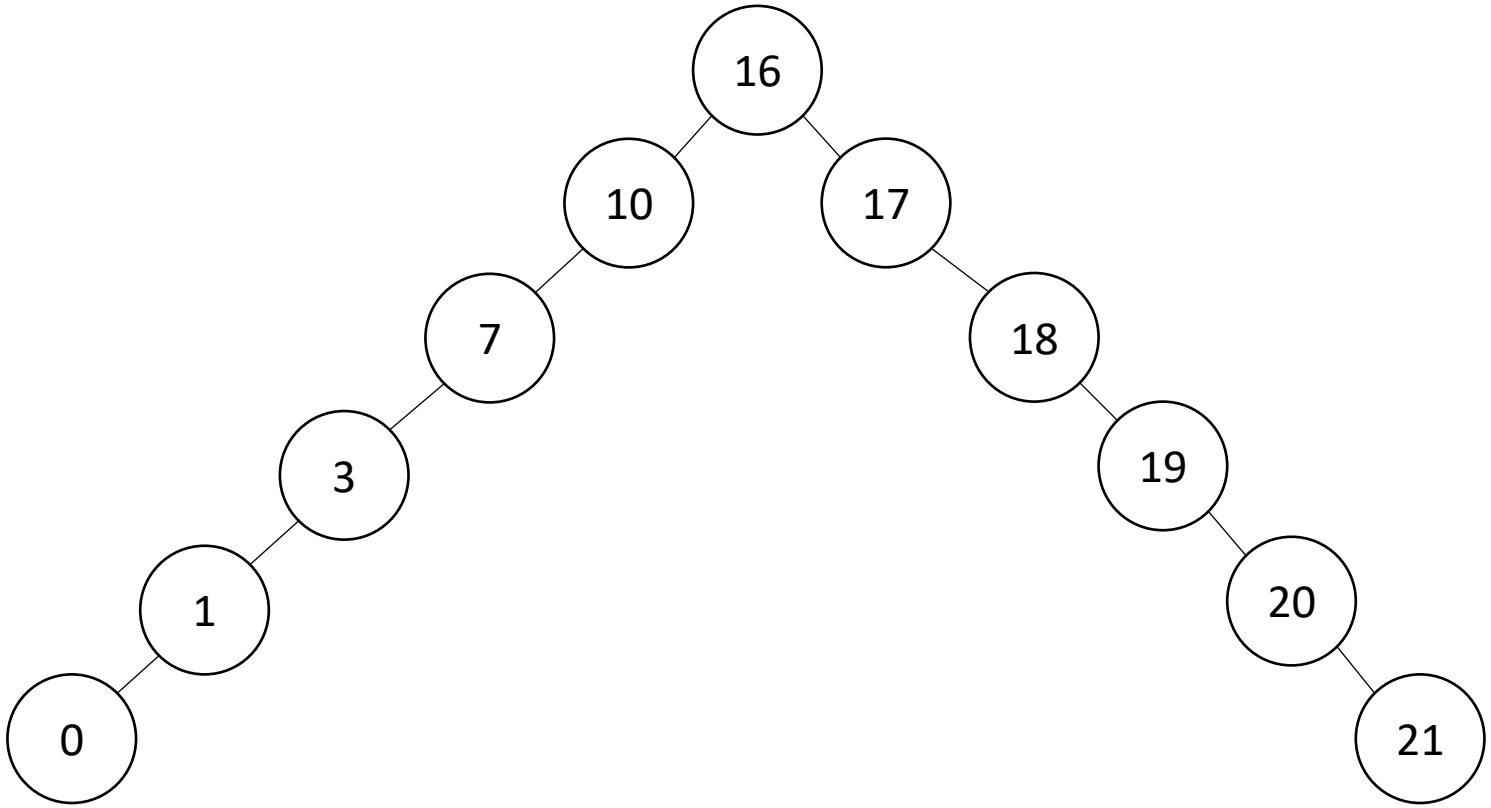
- How can we get a better worst case running time?
  - Add rules about the shape of our BST
- AVL Tree
  - A BST with some shape rules
    - Algorithms need to change to accommodate those

# “Balanced” Binary Search Trees

- We get better running times by having “shorter” trees
- Trees get tall due to them being “sparse” (many one-child nodes)
- Idea: modify how we insert/delete to keep the tree more “full”
  - Encourage Branches!

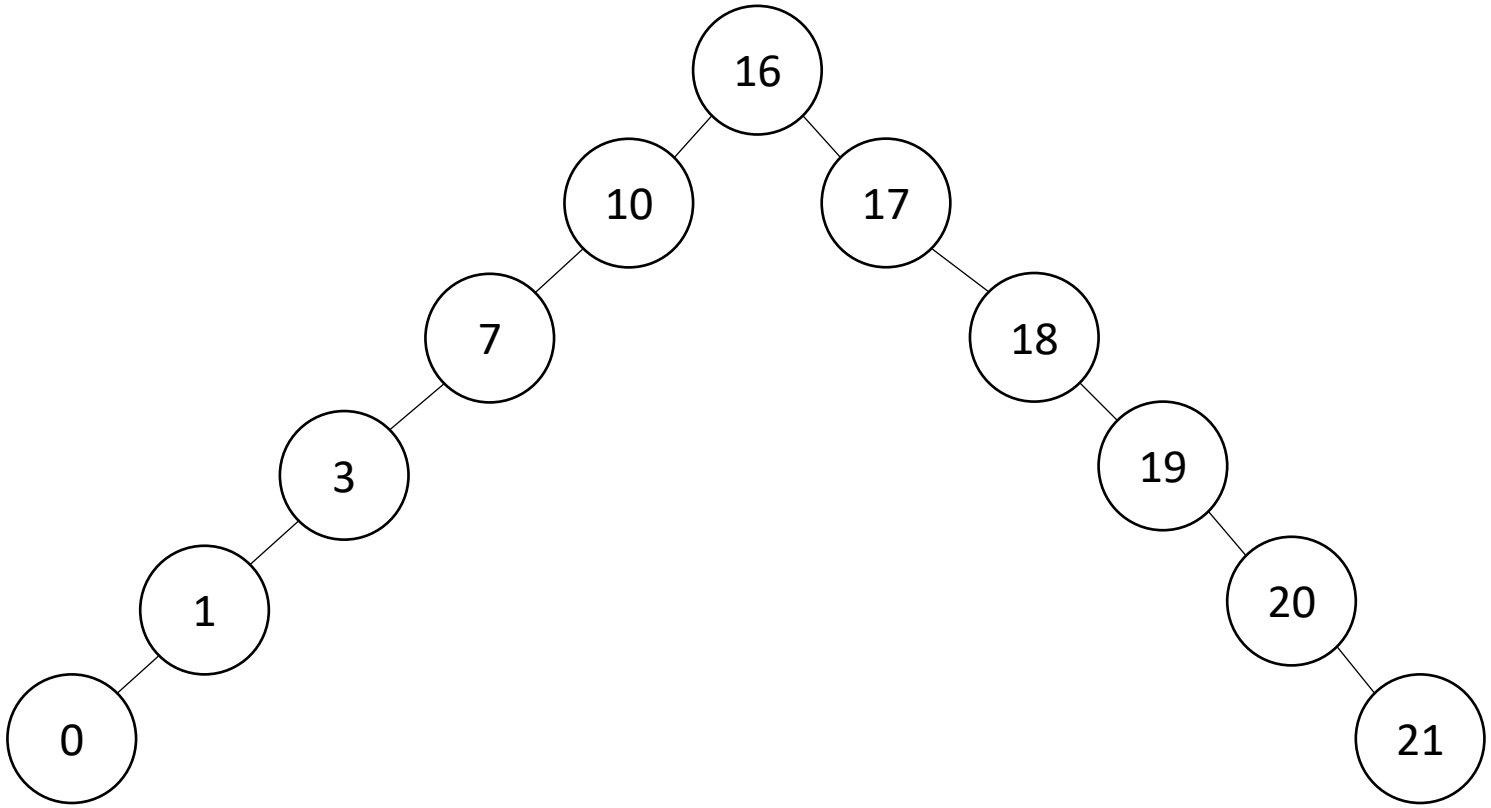
# Idea 1: Both Subtrees of Root have same # Nodes

# Idea 1: Both Subtrees of Root have same # Nodes - Issue



# Idea 2: Both Subtrees of Root have same height

# Idea 2: Both Subtrees of Root have same height - Issue



# Idea 3: Both Subtrees of every Node have same # Nodes

# Idea 3: Both Subtrees of every Node have same # Nodes - Issue

Not all tree sizes are possible!

For example, cannot have a tree of size 3.

# Idea 4: Both Subtrees of every Node have same height

# Idea 4: Both Subtrees of every Node have same height - Issue

Not all tree sizes are possible!

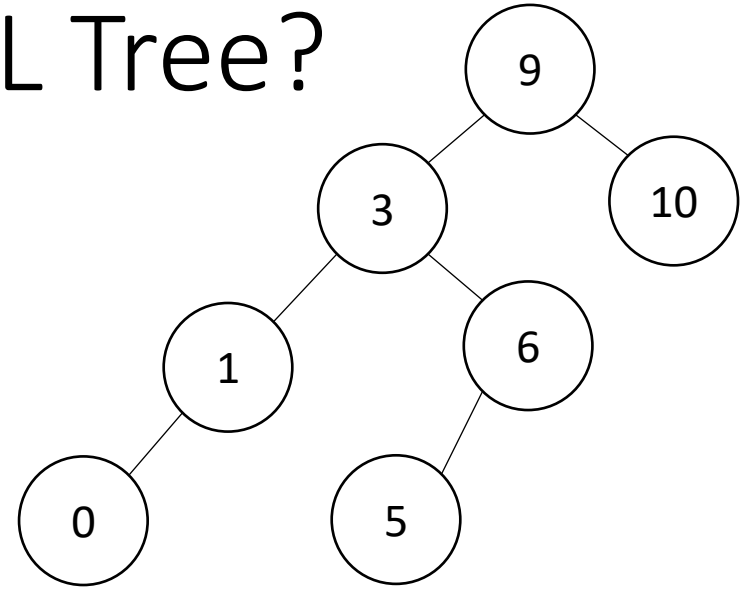
For example, cannot have a tree of size 3.

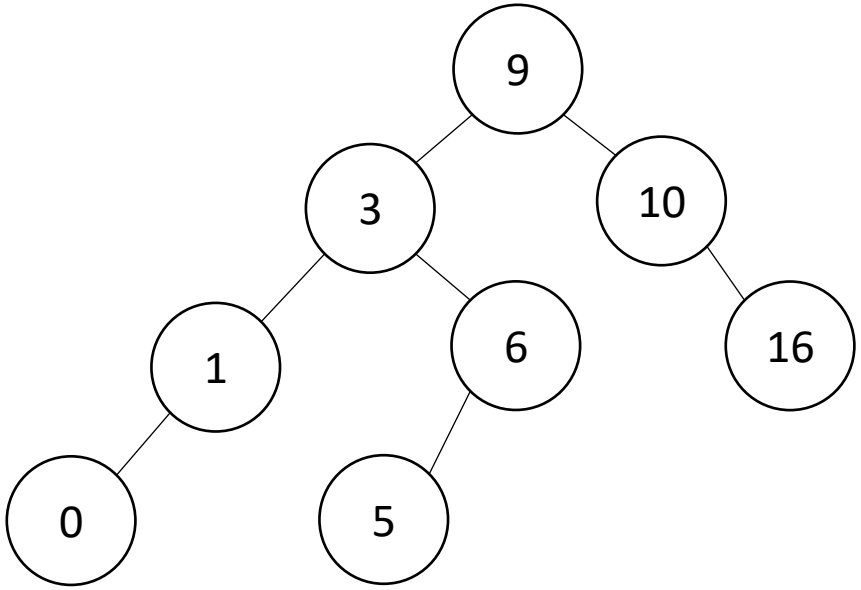
# AVL Tree

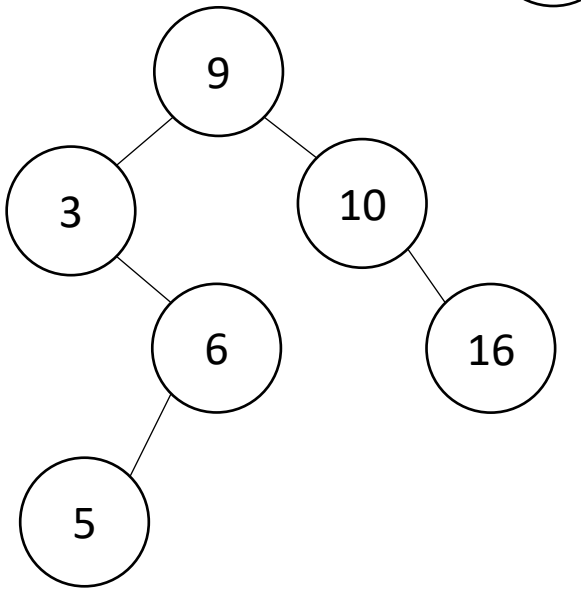
- A Binary Search tree that maintains that the left and right subtrees of every node have heights that differ by at most one.
  - height of left subtree and height of right subtree off by at most 1
  - Not too weak (ensures trees are short)
  - Not too strong (works for any number of nodes)

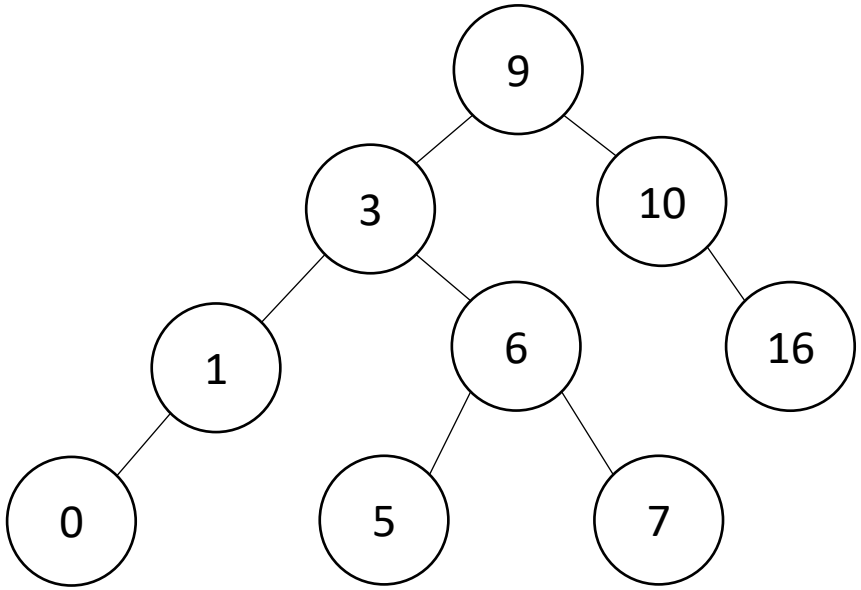
- Idea of AVL Tree:
  - When you insert/delete nodes, if tree is “out of balance” then modify the tree
  - Modification = “rotation”

# Is it an AVL Tree?



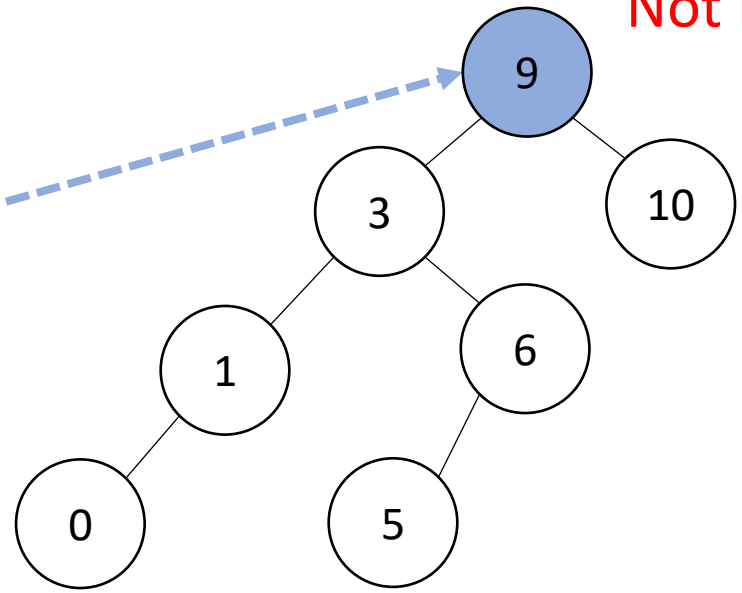



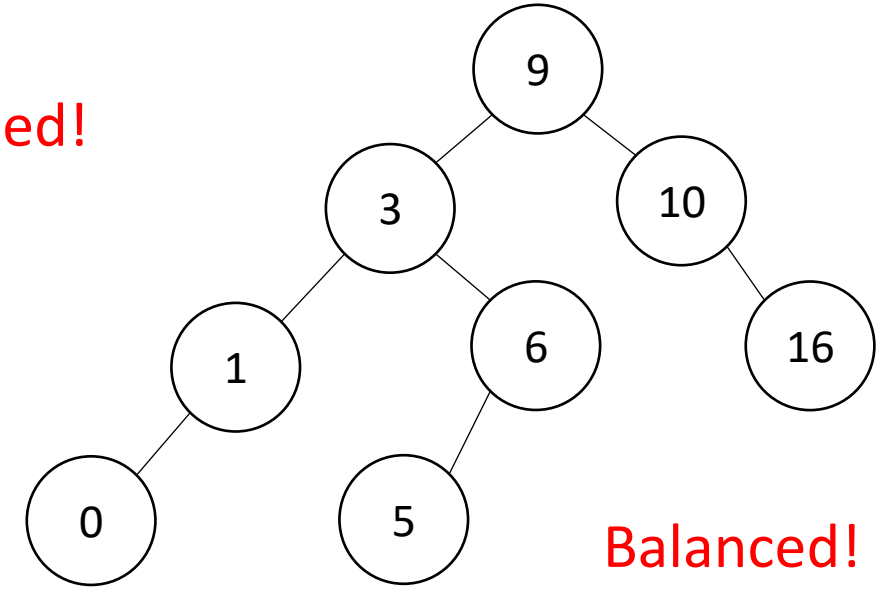



# Is it an AVL Tree? (Answers)

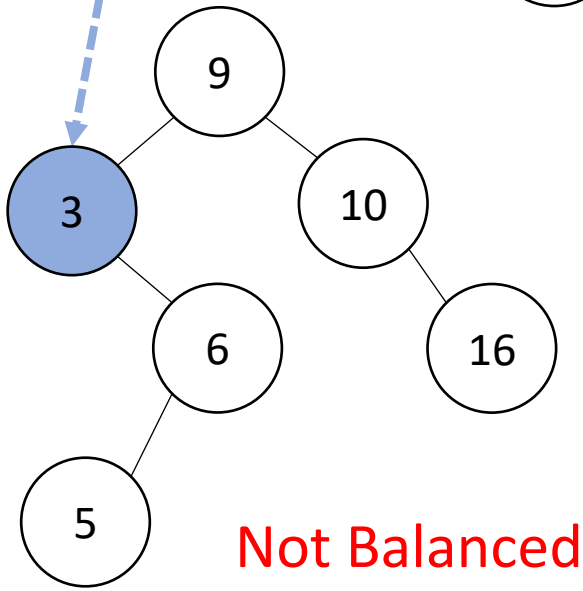
**"Problem" Node**
Its children's heights differ by more than 1



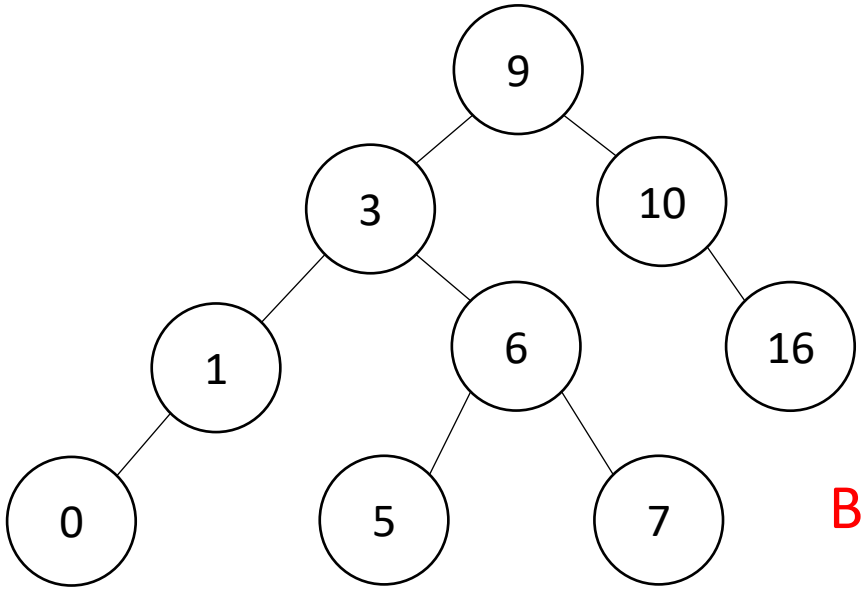

Not Balanced!




Balanced!



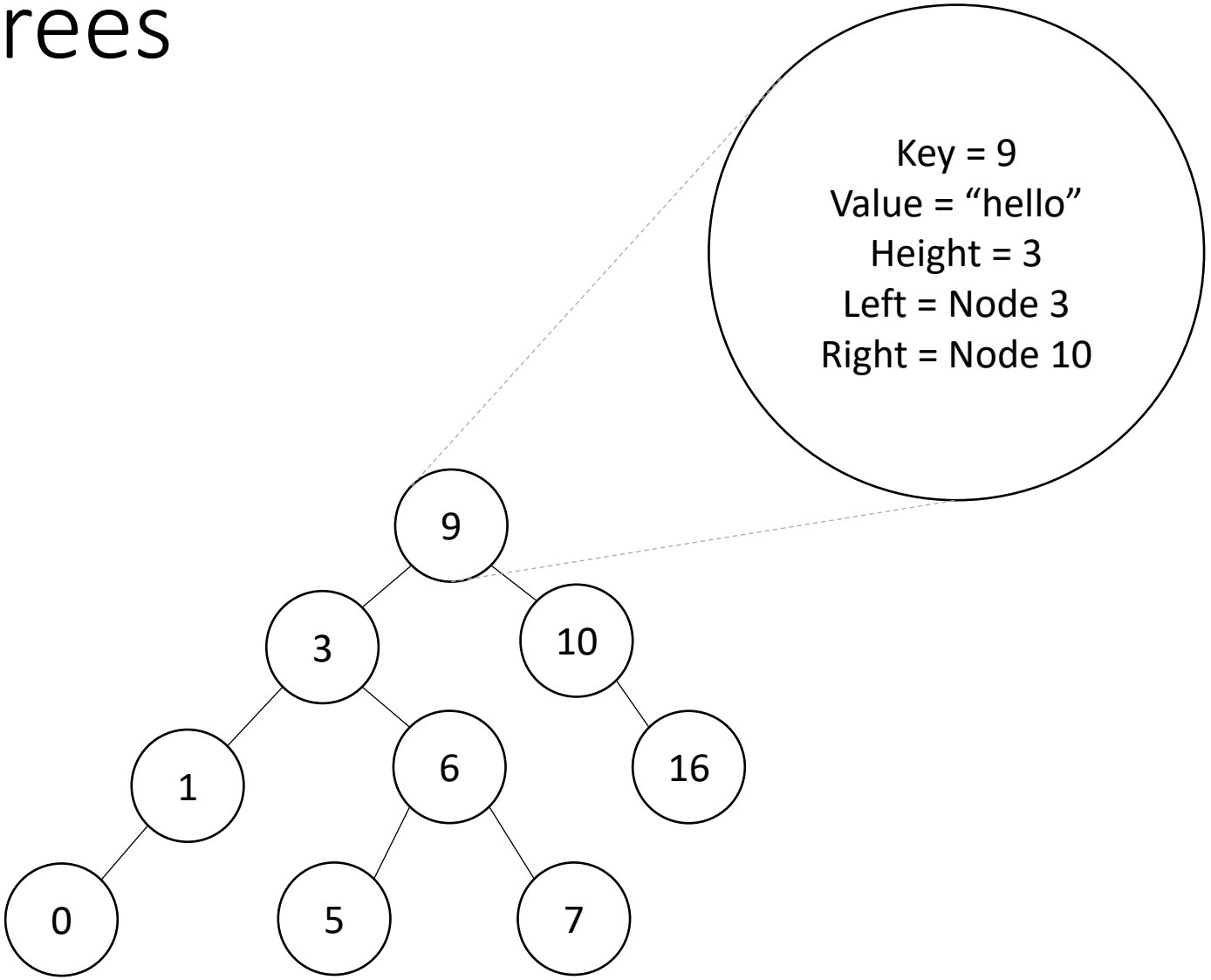

Not Balanced!




Balanced!

# Using AVL Trees

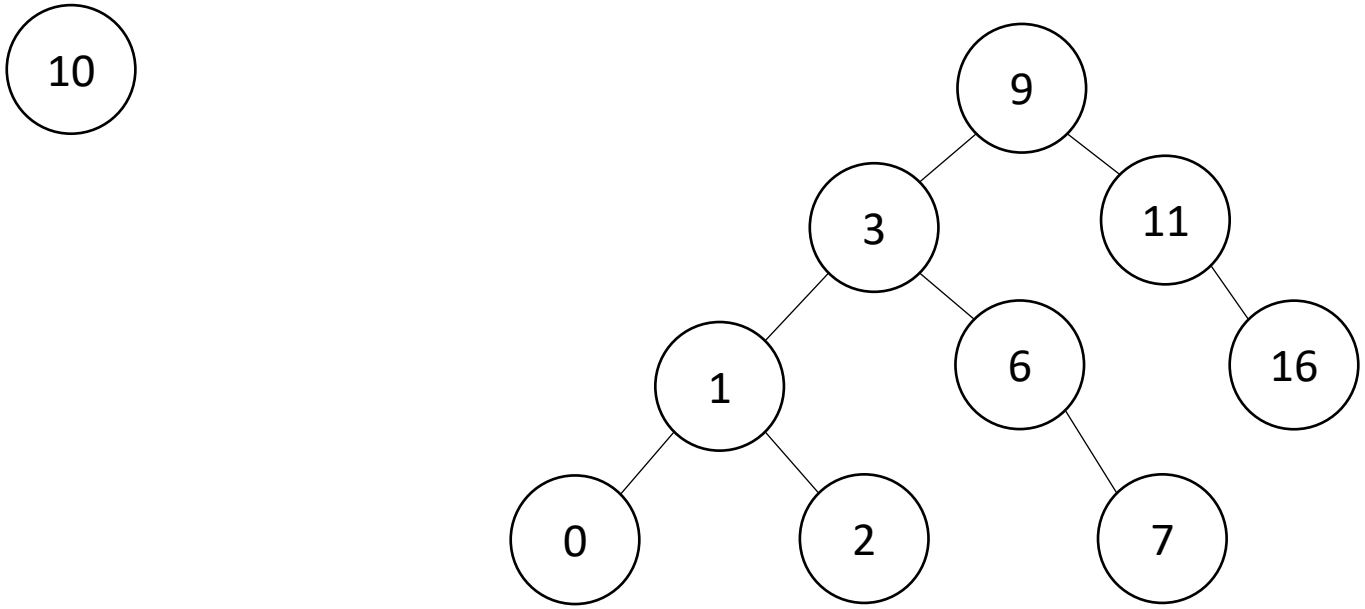
- Each node has:
  - Key
  - Value
  - Height
  - Left child
  - Right child



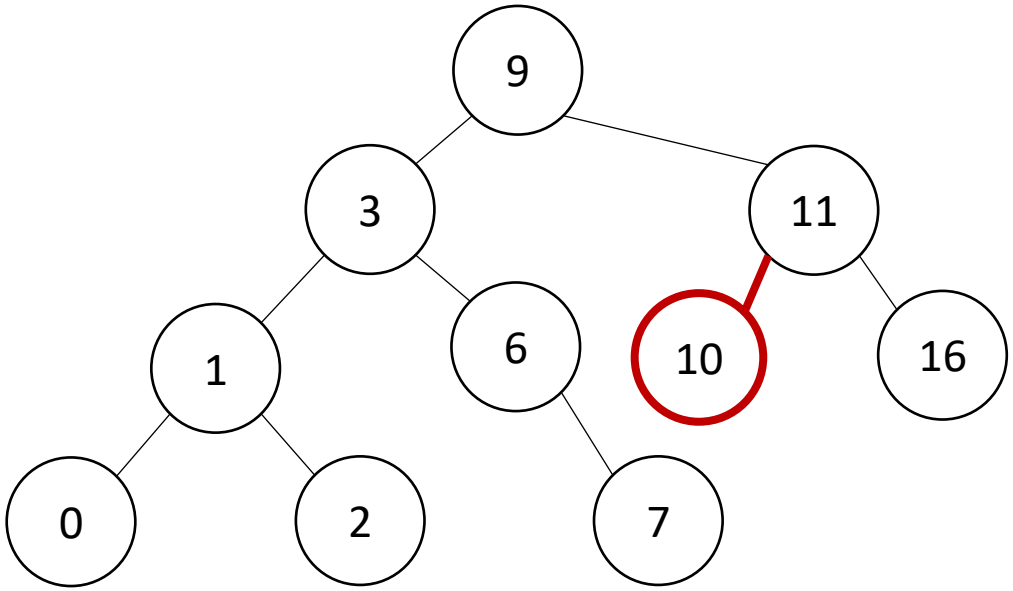
# Inserting into an AVL Tree

- Starts out the same way as BST:
  - “Find” where the new node should go
  - Put it in the right place (it will be a leaf)
- Next check the balance
  - If the tree is still balanced, you’re done!
  - Otherwise we need to do rotations

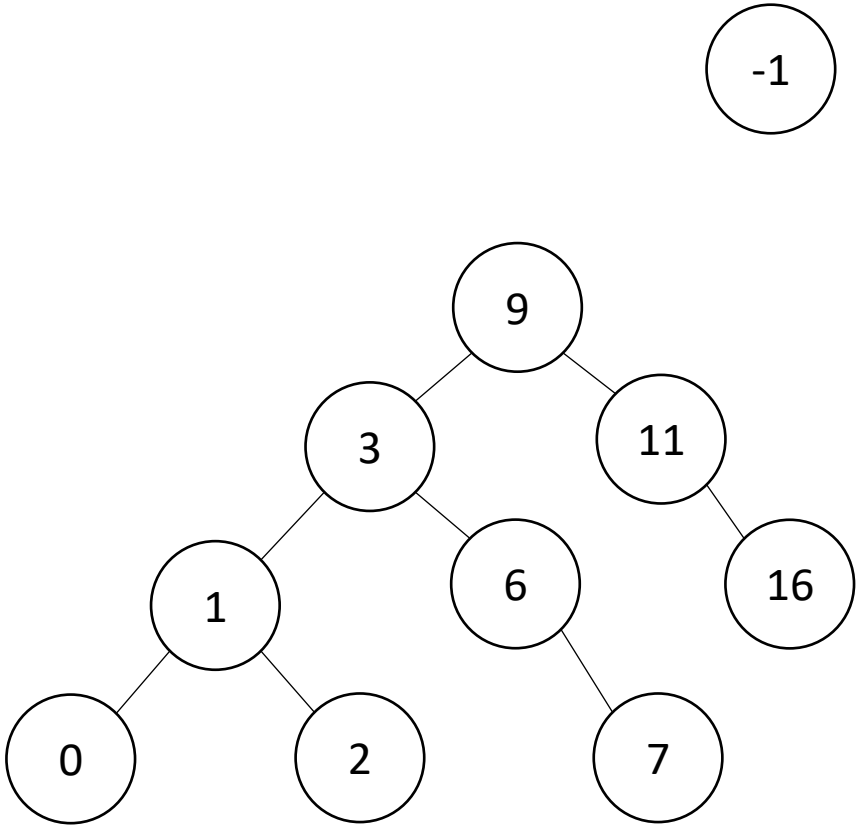
# Insert Example (Insert 10)



# Insert Example (Insert 10) - After

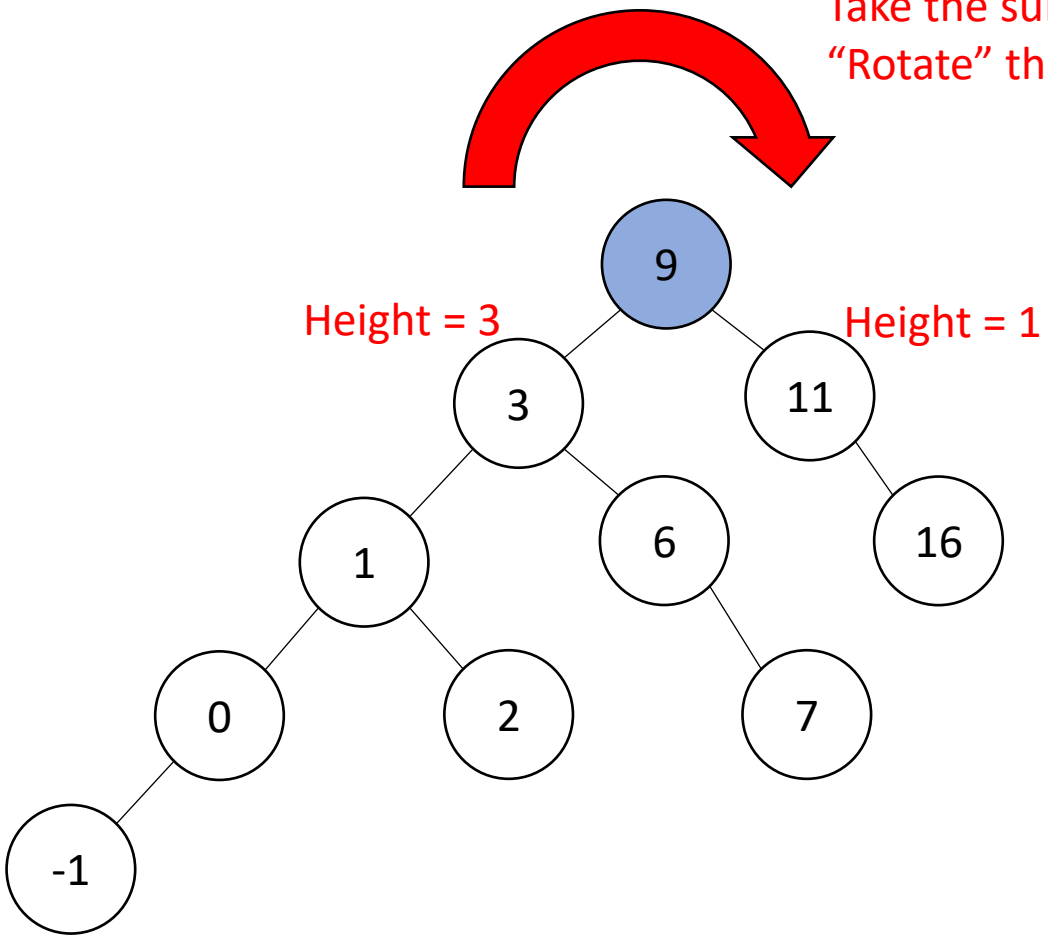


# Insert Example (Insert -1)

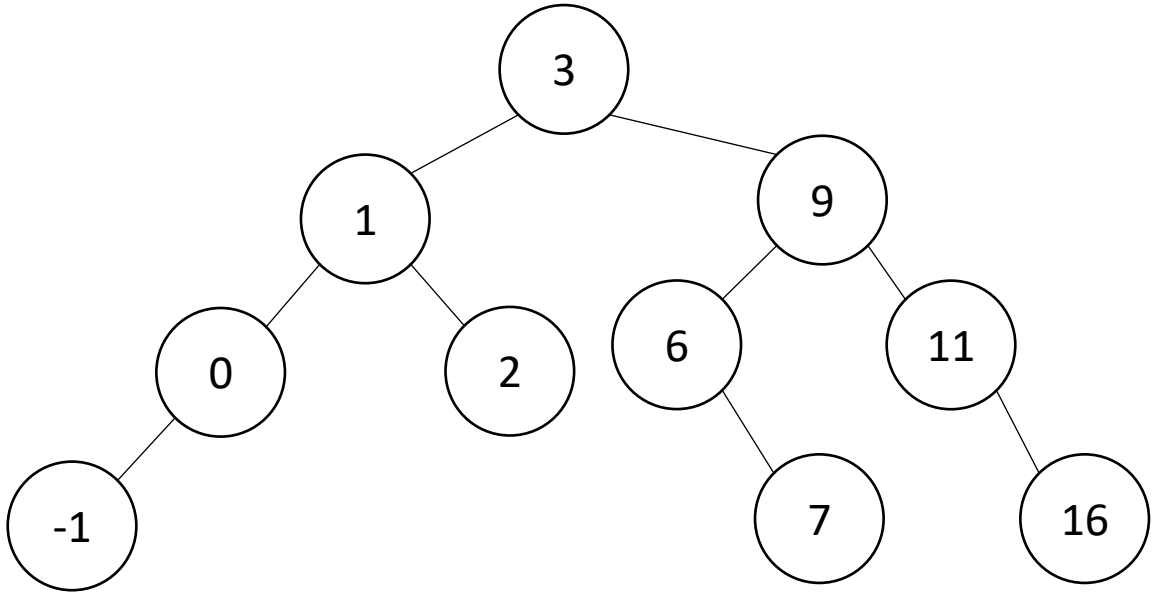


# Not Balanced after Inserting -1

Solution:
Take the subtree starting with the problem node,
“Rotate” that tree to the right

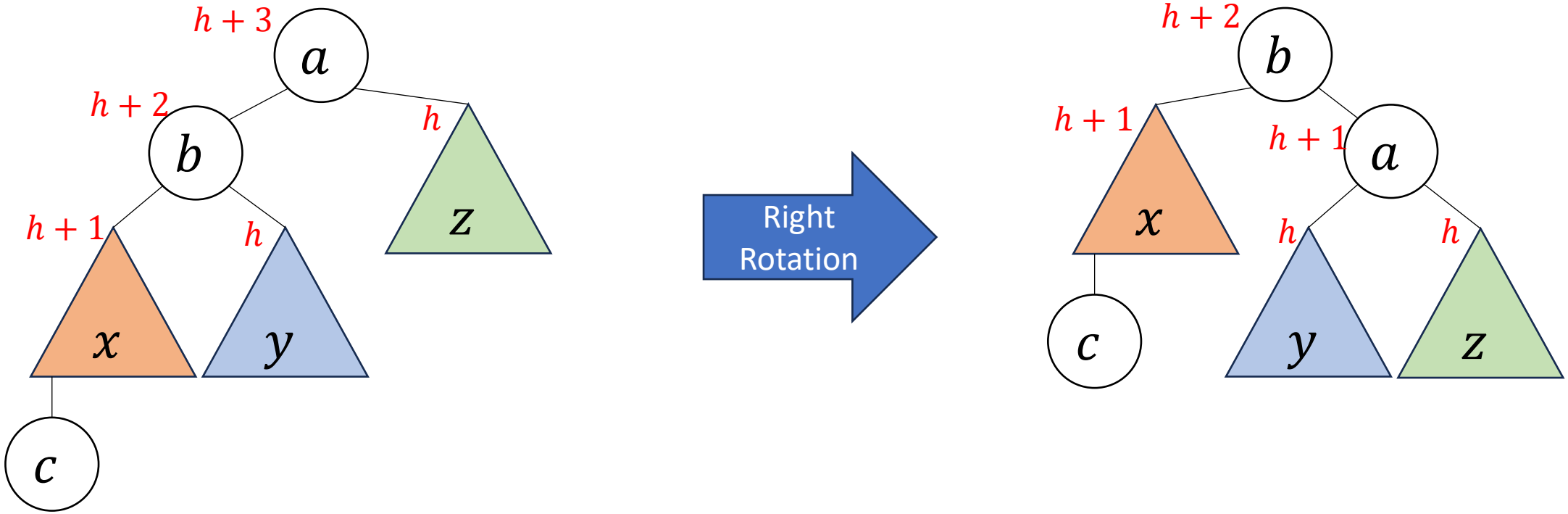


# Final Result of Inserting -1

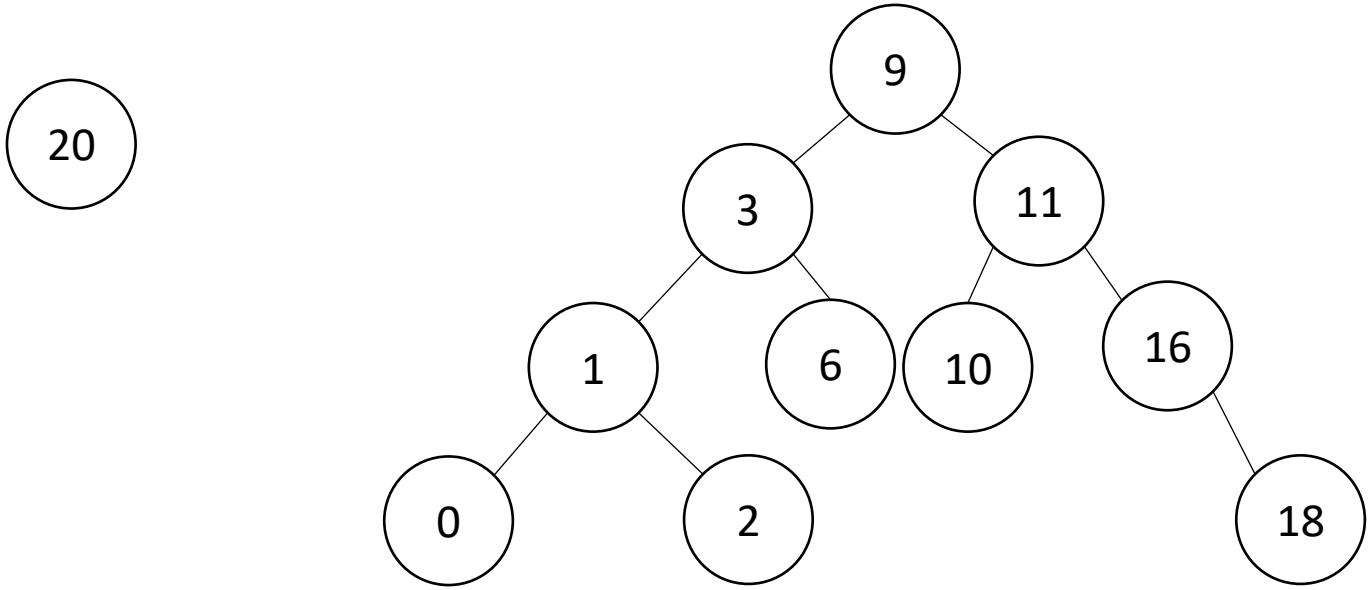


# Right Rotation

- We just inserted $c$, node $a$ is the deepest "problem" node
- Make the left child the new root
- Make the old root the right child of the new
- Make the new root's right subtree the old root's left subtree

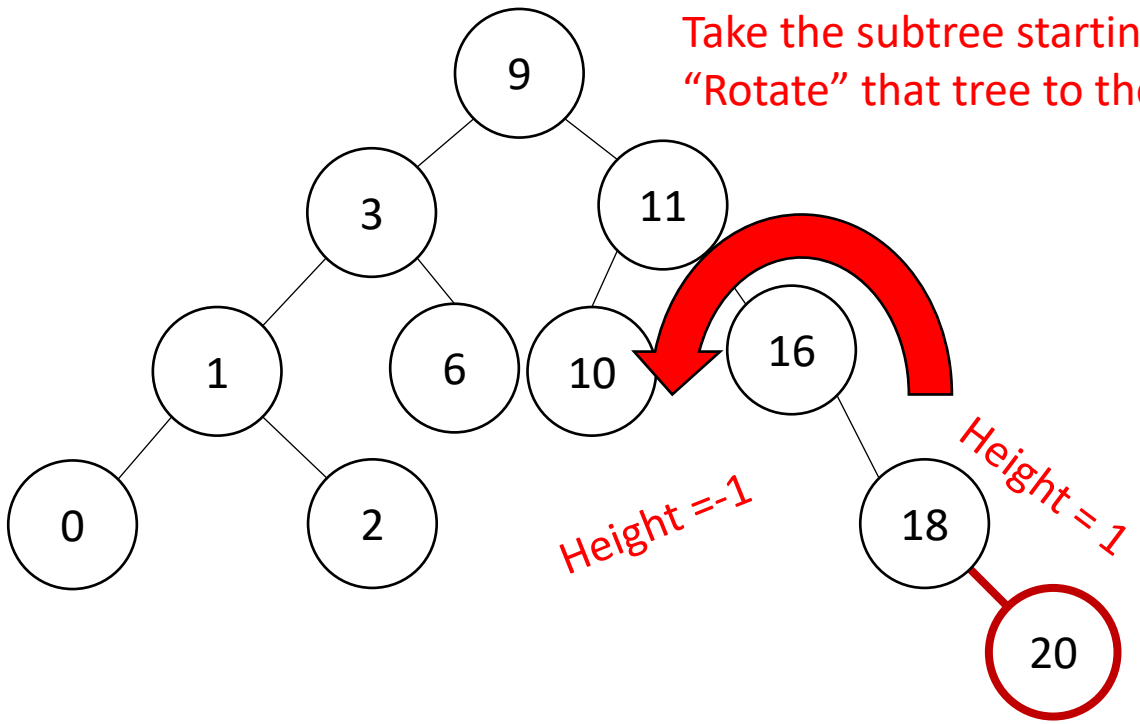


# Insert Example (Insert 20)

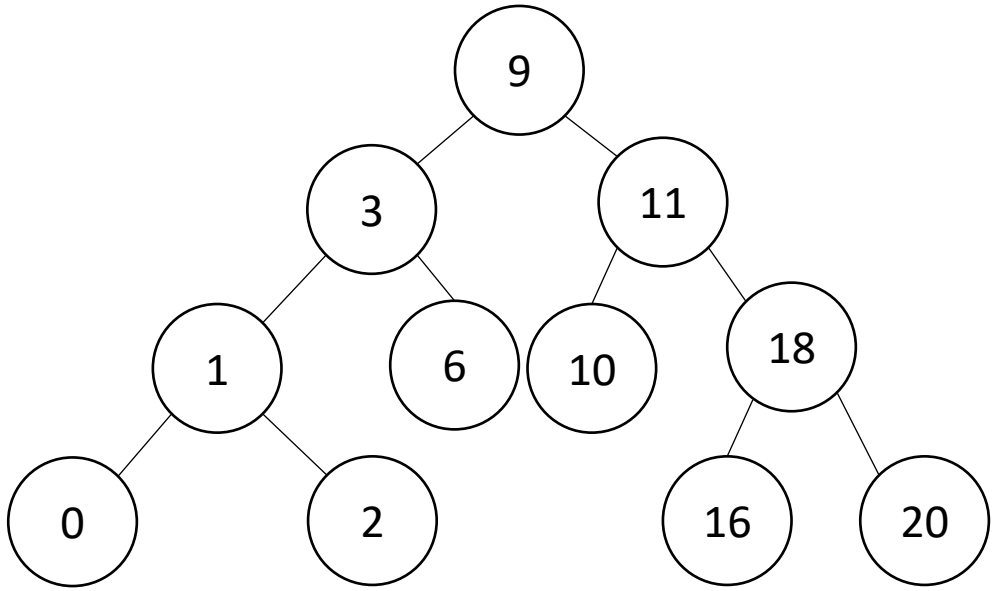


# Not Balanced After inserting 20

Solution:
Take the subtree starting with the problem node,
“Rotate” that tree to the left

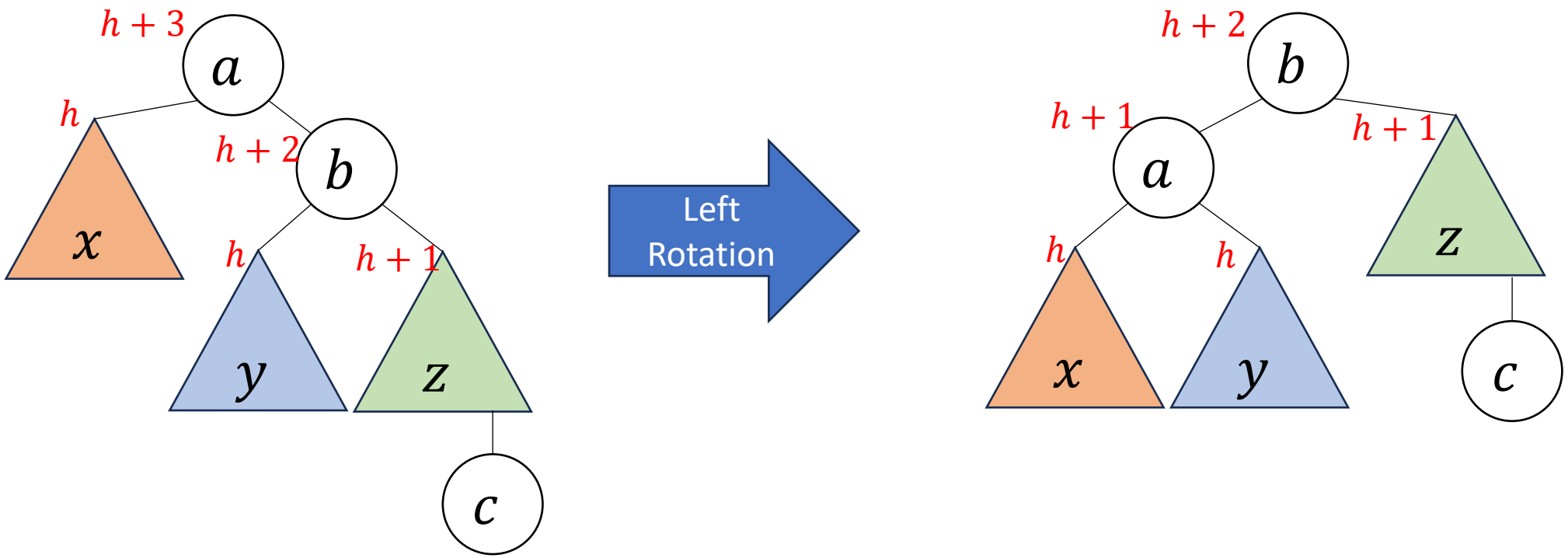


# Final Result of Inserting 20



# Left Rotation

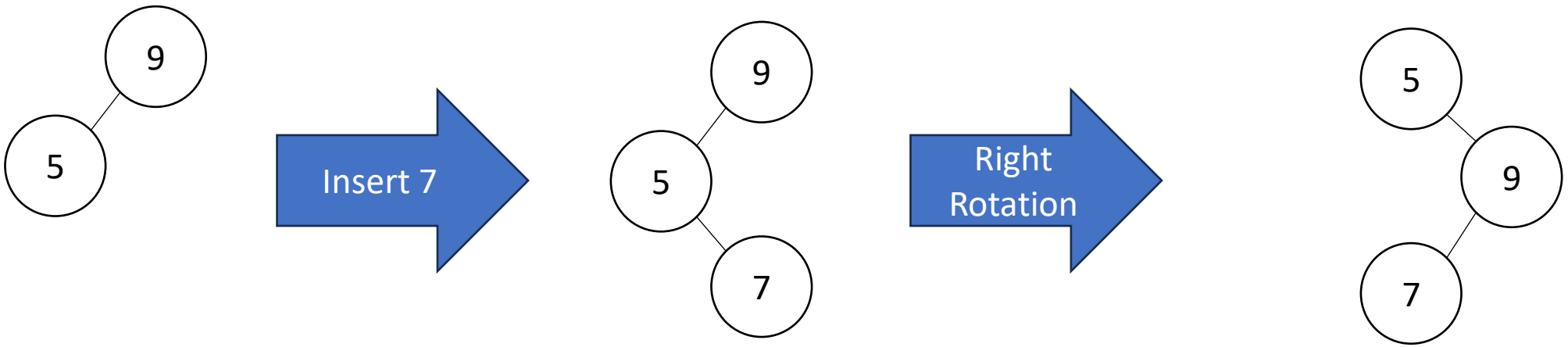
- We just inserted $c$, node $a$ is the deepest "problem" node
- Make the right child the new root
- Make the old root the left child of the new
- Make the new root's left subtree the old root's right subtree



# Insertion Story So Far

- After insertion, update the heights of the node's ancestors
- Check for unbalance
- If unbalanced then at the deepest unbalanced root:
  - If the left subtree was deeper then rotate right
  - If the right subtree was deeper then rotate left

This is incomplete!
There are some cases
where this doesn't work!



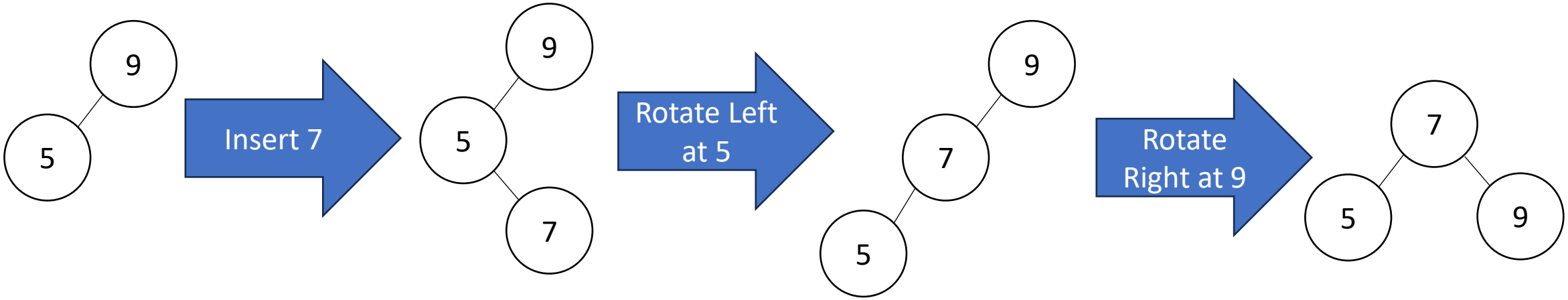
# Insertion Cases

- After insertion, update the heights of the node's ancestors
- Check for unbalance
- If unbalanced then at the deepest unbalanced root:
  - Case LL: If we inserted in the **left** subtree of the **left** child then rotate right
  - Case RR: If we inserted in the **right** subtree of the **right** child then rotate left
  - Case LR: If we inserted into the **right** subtree of the **left** child then ???
  - Case RL: If we inserted into the **left** subtree of the **right** child then ???

Cases LR and RL require 2 rotations!

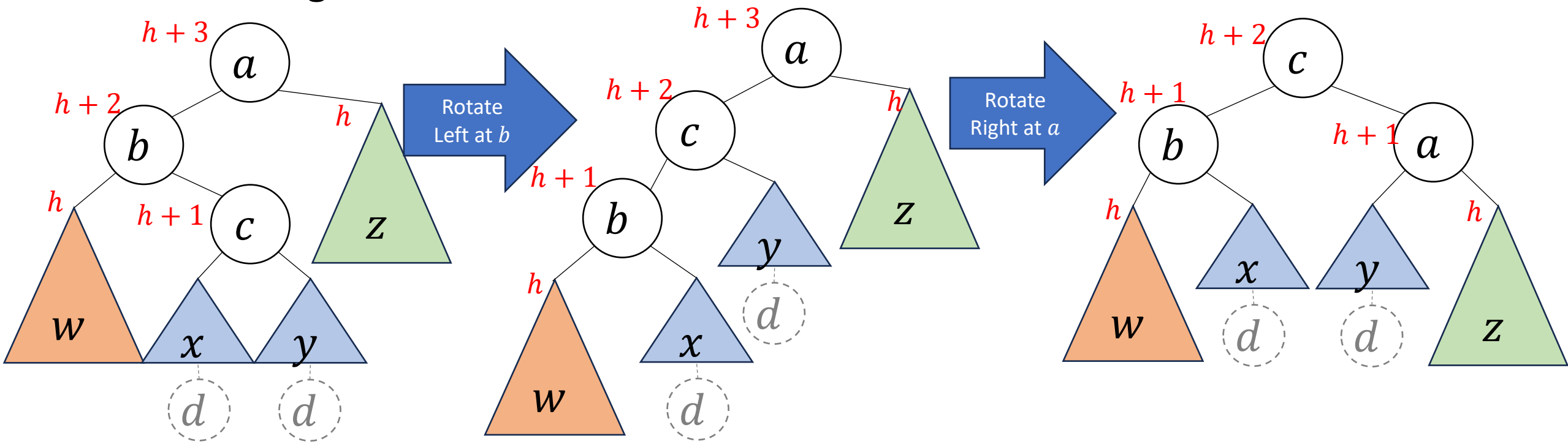
# Case LR

- From deepest problem node:
  - Rotate left at the left child
  - Rotate right at the problem node



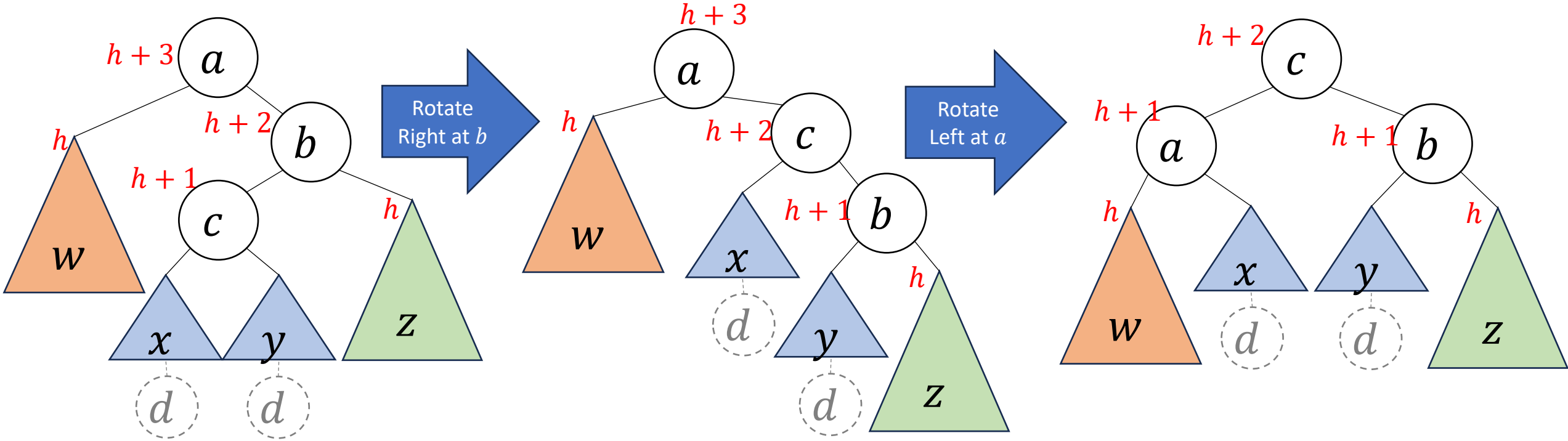
# Case LR in General

- We just inserted $d$, node $a$ is the deepest "problem" node
- Imbalance caused by inserting in the left child's right subtree
- Rotate left at the left child
- Rotate right at the unbalanced node



# Case RL in General

- We just inserted $d$, node $a$ is the deepest "problem" node
- Imbalance caused by inserting in the right child's left subtree
- Rotate right at the right child
- Rotate left at the unbalanced node



# Insert Summary

- After a BST insertion, update the heights of the node's ancestors
- From leaf to root, check if each node is balanced
- If a node is unbalanced then at the deepest unbalanced node:
  - Case LL: If we inserted in the **left** subtree of the **left** child then: rotate right
  - Case RR: If we inserted in the **right** subtree of the **right** child then: rotate left
  - Case LR: If we inserted into the **right** subtree of the **left** child then: rotate left at the left child and then rotate right at the root
  - Case RL: If we inserted into the **left** subtree of the **right** child then: rotate right at the right child and then rotate left at the root
- Done after either reaching the root or applying **one** of the above cases

# Delete Summary

- Tldr: same cases, reverse direction of rotation, may need to repeat with ancestors
- After a BST deletion, update the heights of the node's ancestors
- From leaf to root, check if each node is unbalanced
- If a node is unbalanced then at the deepest unbalanced node:
  - Case LL: If we deleted in the **left** subtree of the **left** child then: rotate left
  - Case RR: If we deleted in the **right** subtree of the **right** child then: rotate right
  - Case LR: If we deleted into the **right** subtree of the **left** child then: rotate right at the left child and then rotate left at the root
  - Case RL: If we deleted into the **left** subtree of the **right** child then: rotate left at the right child and then rotate right at the root
- Continue checking until reach the root